

Customer Identity Resolution for Multi-Source Customer Data

Design and operations based on the current schema and source code

Trieu Nguyen

2026-08-04

Abstract

Customer Identity Resolution (CIR) is the process of linking customer records from multiple systems into a single unified profile. In this implementation, inbound data is first written to a staging table and then processed by a Python resolver driven by metadata rules. The objective is to maintain exactly one master profile per customer within each tenant and domain, while preserving flexibility when new data sources or matching rules are introduced.

1. Scope

This paper is based on components that already exist in the repository:

- PostgreSQL schema in database-init/database-schema.sql
- Resolver module in backend-system/identity_resolution/identity_resolution/resolver.py
- Trigger controller in backend-system/identity_resolution/identity_resolution/trigger_controller.py
- Persona logic in backend-system/identity_resolution/identity_resolution/persona.py

Main Flow (Identity Resolution Context)

Raw Profiles

- > Staging and Validation
- > Identity Resolution Engine (this paper)
 - > Rule Loading from Metadata
 - > Candidate Matching (exact/fuzzy)
 - > Master Profile Merge or Create
 - > Link and Audit Updates
- > Unified Customer Profile
- > Persona Resolution Engine (downstream)
- > Customer360 Master Profile

Stage	Purpose	Output
Raw Profiles	Ingest events and profile traces from source systems	Staging-ready records
Identity Resolution	Match, merge, and link records per tenant/domain	Unified customer profile
Downstream Persona Resolution	Enrich profile with persona intelligence	Persona fields, scores, and embeddings
Customer360 Master Profile	Persist durable customer truth for activation	Single customer view for operations

2. Core Data Model and Data Enrichment Structure

The database stores entities and relationships required by the resolution lifecycle. This section summarizes the core tables, key fields, and their operational role in enrichment.

2.1 Table cdp_raw_profiles_stage (Raw staging table)

This intermediate table acts as the landing zone for raw customer traces from heterogeneous source systems before identity resolution.

Important fields:

- `raw_profile_id` (UUID, primary key): unique identifier for each raw profile event.
- `tenant_id` (UUID): tenant partitioning key for multi-tenant isolation.
- `domain` (TEXT): business domain context (for example: retail, banking, travel).
- `source_system` (TEXT): upstream origin system (for example: POS, AppsFlyer, MoEngage, CRM).
- `external_customer_id` (TEXT): source-local customer identifier.
- `email`, `phone_number`, `national_id` (TEXT): personal identifiers; values can be plaintext or one-way SHA-256 hashes.
- `full_name`, `first_name`, `last_name` (TEXT): source name fields.
- `device_id`, `advertising_id`, `cookie_id`, `push_token` (TEXT): digital/device identifiers for cross-channel resolution.
- `event_name`, `event_time` (TIMESTAMPZ), `event_payload` (JSONB): event semantics, timestamp, and extensible attributes for enrichment.
- `status_code` (SMALLINT, default 1): queue lifecycle (1 ready, 2 processing, 3 processed, 4 failed).
- `processed_at` (TIMESTAMPZ): completion timestamp.

2.2 Table cdp_master_profiles (Master/Golden profile)

This table stores the canonical customer profile after merge, normalization, and enrichment across all sources.

Important fields:

- `master_profile_id` (UUID, primary key): golden profile identifier.
- `tenant_id` (UUID), `domain` (TEXT): strict data scoping dimensions.
- `full_name`, `first_name`, `last_name`, `email`, `phone_number`, `national_id`, `address` (TEXT): consolidated identity fields.
- `secondary_emails`, `secondary_phones` (JSONB): retained alternate contact points to preserve channel reach.
- `external_ids` (JSONB): source-to-external ID map to support reverse synchronization.
- `device_ids`, `advertising_ids`, `cookie_ids` (TEXT[]): deduplicated device identity arrays.
- `push_tokens` (JSONB): push-token map for notification platforms.
- `is_hashed` (BOOLEAN, default FALSE): indicates that sensitive PII values are hashed.
- `persona_name` (TEXT): non-PII identity label generated by deterministic logic or LLM.
- `persona_summary` (TEXT): short behavior narrative used in personalization workflows.
- `persona_embedding` (VECTOR(768)): embedding vector for semantic retrieval and lookalike targeting.
- `updated_at` (TIMESTAMPZ): last update timestamp.
- `first_seen_raw_profile_id` (UUID): lineage reference to the first raw record that created this master profile.
- `source_systems` (TEXT[]): set of contributing source systems.

2.3 Table cdp_profile_links (Profile-link table)

This table stores the active 1-to-N relationship between a master profile and its contributing raw profiles.

Important fields:

- `link_id` (BIGSERIAL, primary key): unique link record identifier.
- `tenant_id` (UUID): tenant partition key.
- `raw_profile_id` (UUID, UNIQUE): one raw profile maps to one active master profile at a time.
- `master_profile_id` (UUID): target master profile.
- `match_score` (NUMERIC): confidence score in range [0.0, 1.0].
- `match_method` (TEXT): algorithm/method label (for example: exact_email, fuzzy_trgm_name).
- `status` (VARCHAR): link state (ACTIVE, HISTORICAL).

2.4 Table cdp_profile_attributes (Attribute metadata registry)

This registry controls matching and consolidation behavior at the attribute level.

Important fields:

- `attribute_internal_code` (VARCHAR, primary key): internal key aligned with staging columns.
- `master_profile_column` (VARCHAR): target column on `cdp_master_profiles`.
- `is_identity_resolution` (BOOLEAN): whether the attribute participates in identity matching.
- `matching_rule` (VARCHAR): matching method (exact, fuzzy_trgm, fuzzy_dmetaphone, none).
- `matching_threshold` (NUMERIC): threshold for fuzzy matching.
- `consolidation_rule` (VARCHAR): merge strategy (most_recent, verified_first, source_priority, non_null, append_distinct, overwrite).

2.5 Table `cdp_identity_index` (Flattened identity index)

This index accelerates exact-match lookups by flattening JSON/array identifiers, reducing scans over complex nested fields under high throughput.

Important fields:

- `identity_index_id` (UUID, primary key): identity index record identifier.
 - `tenant_id` (UUID): tenant scope.
 - `master_profile_id` (UUID): owner master profile.
 - `identifier_type` (VARCHAR): identifier type (for example: email, phone, cookie_id).
 - `identifier_value` (TEXT), `identifier_value_normalized` (TEXT): raw and normalized forms for stable exact matching.
 - `is_blocked` (BOOLEAN): blocks invalid/synthetic values (for example: anonymous, null, void).
-

3. Processing Mechanism

Customer Identity Resolution is orchestrated by `CustomerIdentityResolver` in `resolver.py`. The pipeline is metadata-driven through `cdp_profile_attributes`, which provides strong flexibility and maintainability while preserving strict multi-tenant isolation.

3.1 Detailed Step-by-Step Processing

Each batch executes a sequential seven-step flow:

1. Load active rules

- Query `cdp_profile_attributes` for attributes where `is_identity_resolution = TRUE`, `status = 'ACTIVE'`, and `matching_rule` is defined and not none.

2. Fetch unprocessed staging records

- Query `cdp_raw_profiles_stage` with `status_code = 1`, limited by configured `batch_size` (default: 1,000 records).

3. Set secure tenant context

- For each raw row, execute `SELECT set_config('app.tenant_id', ...)` to activate PostgreSQL RLS and prevent cross-tenant leakage.

4. Build dynamic matching query

- Construct SQL OR conditions from available staged attributes:
 - Device arrays (`device_id`, `advertising_id`, `cookie_id`): `= ANY(array_column)`
 - Source-keyed IDs (`external_customer_id`): `external_ids @> jsonb_build_object(source_system, value)`
 - Exact match: `=`
 - Trigram fuzzy match: `similarity(column, value) >= threshold`
 - Phonetic fuzzy match: `dmetaphone(column) = dmetaphone(value)`

5. Find candidate master profile

- Execute scoped SQL with mandatory partition filters:
 - `WHERE tenant_id = :tenant_id AND domain = :domain AND (dynamic_conditions) LIMIT 1`

6. Merge or create master profile

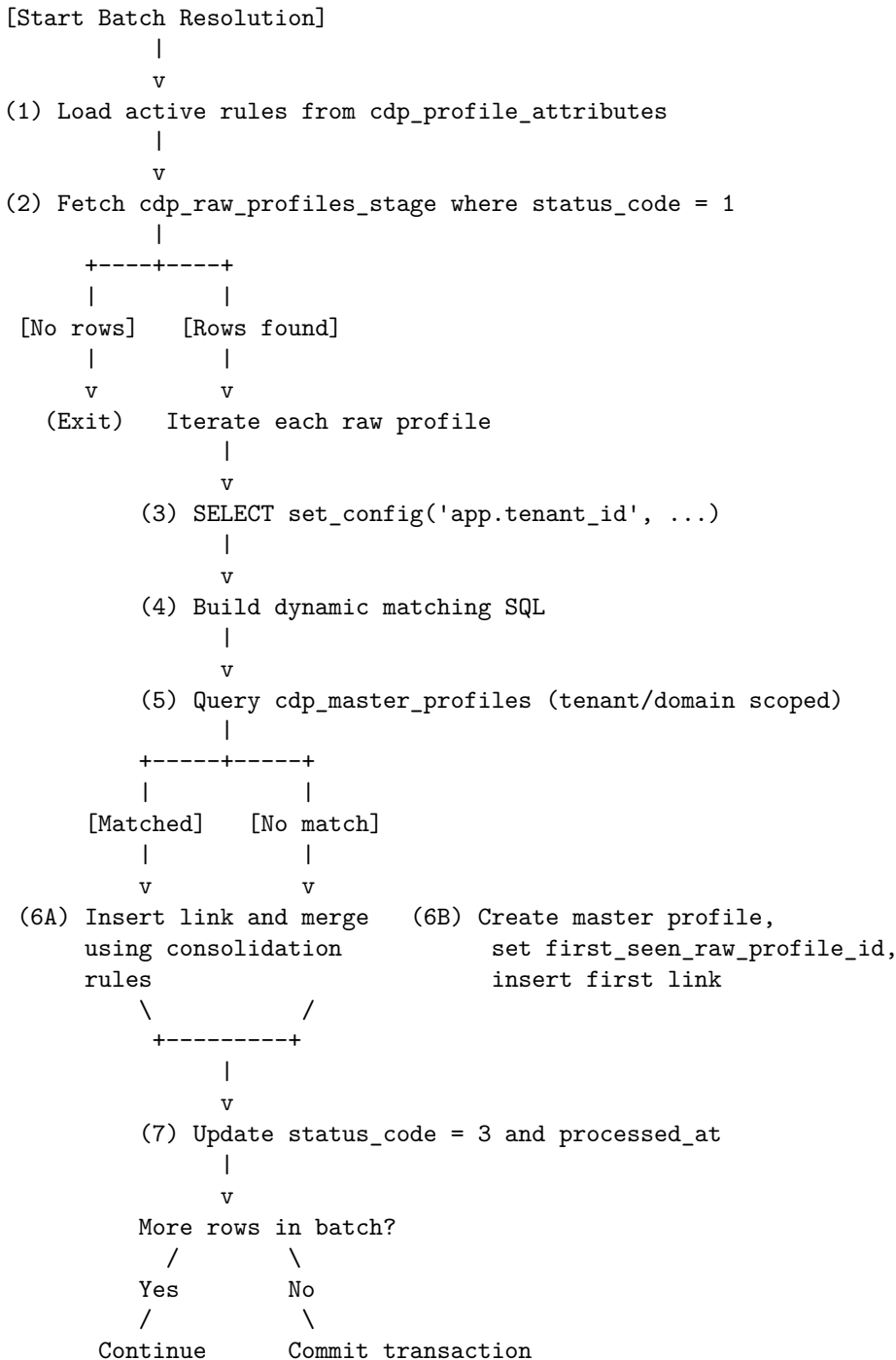
- **If a master profile is found:**
 - Insert link in `cdp_profile_links` with `match_score = 1.0`, `match_method = 'DynamicMatch'`
 - Merge scalar attributes according to configured `consolidation_rule`
 - Union/append arrays and JSON maps (`source_systems`, `external_ids`, `push_tokens`, and related fields)
 - Detect hashed PII and, when needed, set `is_hashed = TRUE` and generate `persona_name`
- **If no master profile is found:**

- Create new row in cdp_master_profiles, preserving first_seen_raw_profile_id
- Insert first link with match_method = 'NewMaster'

7. Mark processed and commit

- Update staging row to status_code = 3 and set processed_at = NOW()
- Commit transaction at batch end; on failure, rollback

3.2 Resolution Execution Flow



3.3 Matching Rules and Data Examples

The system supports four rule families controlled by matching_rule in cdp_profile_attributes.

1. exact rule Used for stable high-precision identifiers. Matching uses = or array containment semantics where applicable.

• Inbound staging:

- email: nguyena@gmail.com
- phone_number: +84901234567

- **Existing master:**
 - email: nguyena@gmail.com
 - phone_number: +84901234567
- **Outcome:** deterministic match; row is linked to existing master profile.

2. fuzzy_trgm rule Used for typo-tolerant text fields such as names or addresses. It relies on PostgreSQL pg_trgm similarity with configurable threshold.

- **Inbound staging:**
 - full_name: Nguyen Van A
 - address: 123 Duong Le Loi, Phuong 1, Quan 1, TPHCM
- **Existing master:**
 - full_name: Nguyen Van A
 - address: 123 Le Loi, P.1, Q.1, TP HCM
- **Outcome:** similarity exceeds threshold (for example, 0.65), so records are treated as same customer.

3. fuzzy_dmetaphone rule Used for phonetic matching of names with small spelling variations.

- **Inbound staging:** first_name = Smith
- **Existing master:** first_name = Smyth
- **Outcome:** both map to equivalent phonetic code, producing a successful match.

4. none rule Attribute is excluded from identity matching and used only for post-match enrichment.

4. Sensitive Data Handling (Privacy and Hashed PII)

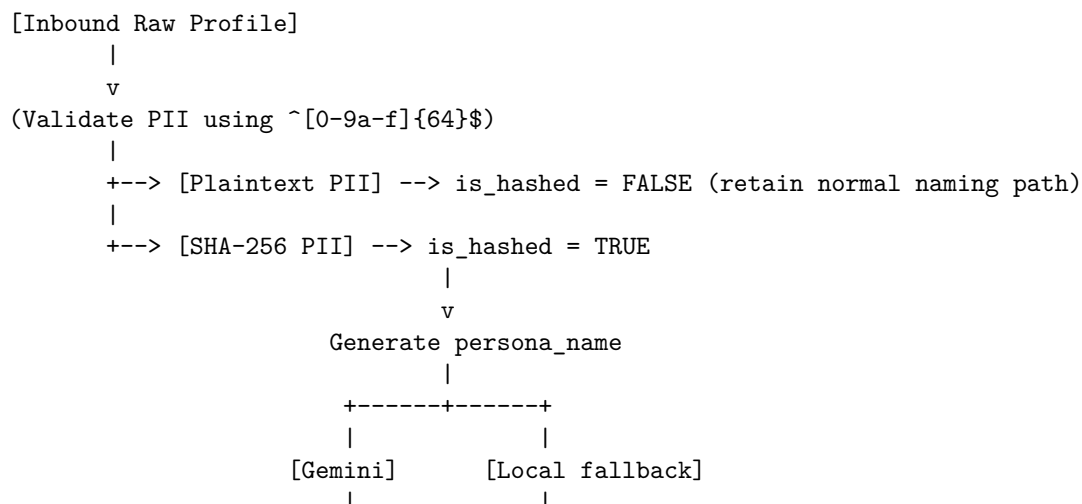
To align with personal-data regulations and ad-tech hashed matching patterns, the engine supports one-way SHA-256 PII payloads (64 hex characters).

4.1 Detection and Persona Name Generation

When PII fields (full_name, email, phone_number, national_id) arrive as hashes, plaintext display is impossible. Therefore, persona.py triggers a non-PII naming path:

- Automatic detection**
 - Regex `^[0-9a-f]{64}$` identifies hashed values
 - `is_hashed = TRUE` is set on `cdp_master_profiles`
- Persona generation**
 - **LLM path (Gemini):** if `GOOGLE_GENAI_API_KEY` is configured, generate memorable non-PII names from non-sensitive context (domain, channel, source)
 - **Deterministic offline fallback:** if no API key/network, derive a stable label from anchor identifiers (device_id, advertising_id, and similar), with a 6-character hash suffix

4.2 Sensitive Data Sequence Flow



```

+-----+-----+
|
|
v
[Master Profile Output]
persona_name = "Digital Banking User #a1b2c3"

```

4.3 Example Data (Hashed vs Plaintext)

Attribute	Plaintext Input	Hashed Input (SHA-256)	Stored in Master
full_name	Nguyen Van An	9f86d08188...15b0f00a08	9f86d08188...15b0f00a08
email	an.nguyen@gmail.com	d081884c7d...c15b0f00a08	d081884c7d...c15b0f00a08
is_hashed	FALSE	TRUE	TRUE
persona_name	NULL	auto-generated	Savvy Retail Shopper #4f2a9c

5. Triggering and Throttling

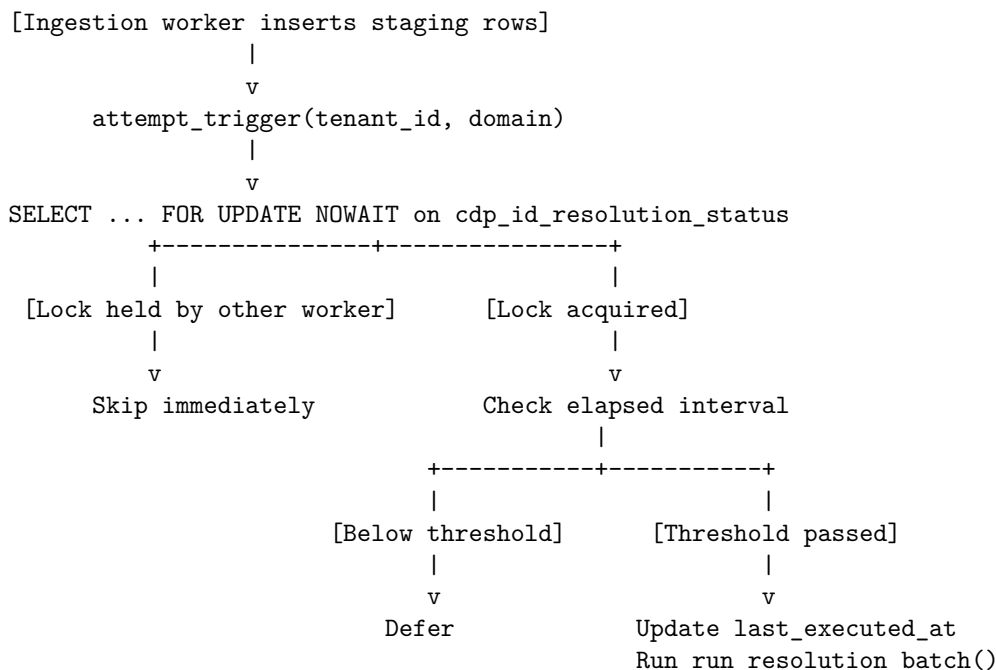
To sustain performance under continuous ingestion, the architecture combines near-real-time triggering with periodic batch processing.

5.1 Row-Lock Throttling Mechanism

The system avoids direct PL/pgSQL triggers to reduce lock contention. Instead, the ingestion worker invokes `IdentityResolutionTrigger.attempt_trigger()` after writes:

1. Runtime state is controlled by single-row table `cdp_id_resolution_status`.
2. `SELECT ... FOR UPDATE NOWAIT` is used:
 - If lock is held by another worker, current trigger call exits immediately (no blocking).
 - If elapsed time from prior run is below `throttle_seconds` (default: 5 seconds), trigger is deferred for micro-batching.
3. If conditions pass, worker updates `last_executed_at` and runs `run_resolution_batch()`.

5.2 Trigger and Throttling Control Flow



5.3 Operational Status Example

Table cdp_id_resolution_status:

id	last_executed_at	updated_at	Runtime Interpretation
TRUE	2026-08-04 10:00:00+07	2026-08-04 10:00:00+07	Run at 10:00:02 is throttled (< 5s), run at 10:00:06 is accepted

6. Operational Best Practices

1. Idempotency and integrity

- Process only staging rows with status_code = 1
- Move processed rows to status_code = 3 with processed_at = NOW()
- UNIQUE(tenant_id, raw_profile_id) on cdp_profile_links prevents duplicate links on retries

2. Tenant and domain security boundaries

- All generated SQL enforces WHERE tenant_id = :tenant_id AND domain = :domain
- This prevents cross-tenant and cross-domain leakage

3. Data normalization before matching

- Phone values should follow E.164 format (for example, +84901234567)
- Email values should be lowercased and trimmed before staging

4. Failure handling and queue monitoring

- Alert on excessive status_code = 1 backlog
- Alert on recurrent status_code = 4 failures

7. Conclusion and Solution Evaluation

The Customer Identity Resolution implementation in this Customer 360 architecture is metadata-driven, tenant-aware, and operationally practical. It supports both near-real-time micro-batching and scheduled high-throughput batch execution, while producing stable golden profiles without introducing database bottlenecks.

7.1 Use Case Application Matrix

Business Use Case	Core Technical Requirement	C360 CIR Resolution Mechanism
Cross-channel identity stitching	Link Web/App/POS/CRM/Ads records	Exact/fuzzy matching on contact identifiers plus accumulation of device_ids, cookie_ids, advertising_ids, and external_ids
Multi-tenant and multi-domain governance Privacy-preserving hashed PII and ad-tech matching	Strict data partition by tenant/domain Support SHA-256 payloads without plaintext exposure	Hard SQL scope filters and PostgreSQL RLS Hash detection, is_hashed flagging, non-PII persona naming via Gemini/local fallback, optional embedding enrichment
Dirty data and typo tolerance	Resolve identities under spelling variations	fuzzy_trgm and fuzzy_dmetaphone rules with configurable thresholds
Attribute conflict handling	Resolve contradictory source values	Field-level consolidation_rule: verified_first, most_recent, source_priority, append_distinct, and others

7.2 Detailed Comparison with Commercial Alternative (Twilio Segment Unify vs Native C360 CIR)

Comparison Dimension	Native Customer 360 CIR Engine	Twilio Segment Unify (Personas)
Deployment and governance	Native PostgreSQL 16 (on-prem/private cloud), full infrastructure and data control, no vendor lock-in	Managed SaaS, data traverses vendor infrastructure and governance model
Matching engine	Hybrid matching: exact + trigram fuzzy + phonetic fuzzy, metadata-driven	Primarily deterministic identity graph around userId/anonymousId; limited native fuzzy matching
Multi-tenant isolation	Database-level isolation with RLS and strict tenant/domain scoping	Workspace/source-level separation; can become costly at large tenant counts
Consolidation strategy	Per-field configurable merge policies (most_recent, verified_first, source_priority, append_distinct, overwrite)	Simpler trait-priority/last-write defaults
Privacy and AI enrichment	Native SHA-256 handling, persona name generation with Gemini, embedding generation for semantic use cases	Usually requires extra custom transformations/functions outside core flow
Total cost of ownership	Primarily infra and worker cost; no MTU licensing growth curve	MTU-based pricing; cost scales rapidly with user volume
Ingestion latency and throughput	Flexible near-real-time micro-batching or scheduled batch	Strong event-driven SaaS near-real-time model

7.3 Final Summary

By combining metadata-driven flexibility, secure hashed-PII handling, and strict tenant isolation at the database layer, the Native C360 CIR Engine provides a strong and cost-efficient foundation for large-scale identity unification. The design is particularly suitable for organizations that require full data ownership, transparent matching logic, and low long-term TCO while still supporting advanced personalization and analytics workflows.